TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

LABORATORY REPORT

Lab 11: The Race and the Semaphore

Operating Systems

Student: Gafenco Victor

Group: FAF-241

Instructor: Assistant lecturer, Patricia Reițman

Chișinău - 2026

CONTENTS

Abstract **3**

Introduction **4**

1 Part 1: See the race **5**

1.1 Source code and execution of `race.c` 5

1.2 Observations for Part 1 5

2 Part 2: Fix it: mutex, then semaphore **7**

2.1 2.1 A mutex around `counter++` (`mutex.c`) 7

2.2 2.2 The same lock, made from a semaphore (`sem.c`) 7

2.3 2.3 The price of the lock 8

2.4 Observations for Part 2 9

3 Part 3: A semaphore as a signal **10**

3.1 Source code and execution of `order.c` 10

3.2 Observations for Part 3 11

4 Part 4: A semaphore as a counter: the parking lot **12**

4.1 Source code and execution of `parking.c` 12

4.2 Observations for Part 4 and Optional Atomic Counter 13

Conclusions **14**

ABSTRACT

The goal of this laboratory is to observe a data race when two threads concurrently update a shared counter without synchronization, eliminate the race using both a POSIX mutex and a POSIX semaphore, and apply the same semaphore primitive in three distinct roles: as a binary lock, as an ordering signal, and as a resource counter. In Part 1, an unprotected counter incremented one million times by each of two threads (`race.c`) loses hundreds of thousands of additions because `counter++` compiles into three non-atomic steps (load, add, store). In Part 2, wrapping the increment in a mutex (`mutex.c`) or a semaphore initialized to 1 (`sem.c`) guarantees the exact result of `2 000 000`, whereas initializing the semaphore to 2 reintroduces the race. In Part 3, a semaphore initialized to 0 (`order.c`) forces a reader thread to wait until a writer thread finishes writing a message. In Part 4, a semaphore initialized to 3 (`parking.c`) limits six car threads to at most three concurrent occupants in a parking lot. Finally, the optional C11 atomic version (`atomic.c`) achieves the exact sum without locks using a single indivisible CPU instruction.

Keywords: Linux, pthreads, race condition, critical section, mutex, POSIX semaphore, signaling, counting semaphore, C11 atomics

INTRODUCTION

Goal of the laboratory (Scopul lucrării)

The purpose of this laboratory work is to observe how concurrent threads racing on shared memory lose updates, fix the race using POSIX synchronization primitives, and use a semaphore in three fundamental ways: as a mutual-exclusion lock, as an inter-thread ordering signal, and as a bounded resource counter. Specifically, the scope and objectives of the laboratory are:

1. **Part 1 (See the race):** Compile and run `race.c` (without `-O2` so that `counter++` remains in memory), observe the non-deterministic lost additions when two threads each add 1 to `counter` one million times, and explain the three CPU steps behind `counter++`.
2. **Part 2 (Fix it: mutex, then semaphore):** Protect `counter++` first with a `pthread_mutex_t` (`mutex.c`) and then with a binary `sem_t` initialized to 1 (`sem.c`), test what happens when the semaphore is initialized to 2, and measure the execution-time cost of locking.
3. **Part 3 (A semaphore as a signal):** Enforce execution order between a writer thread A and a reader thread B using a semaphore initialized to 0 (`order.c`), and observe what happens if its initial value is changed to 1.
4. **Part 4 (A semaphore as a counter: the parking lot):** Use a counting semaphore initialized to 3 alongside a mutex for printing (`parking.c`) so that six car threads share three parking places, and observe how setting `PLACES` to 1 turns the counting semaphore back into a binary lock.
5. **Optional (No lock at all):** Use `atomic_fetch_add` from `<stdatomic.h>` (`atomic.c`) to perform each increment as one indivisible CPU instruction and compare its speed with the mutex and semaphore versions.

All programs were compiled on Linux (openSUSE Tumbleweed, x86_64) with `gcc -Wall -pthread`. Complete execution transcripts are stored in `evidence/` and `report.md`.

1 PART 1: SEE THE RACE

1.1 Source code and execution of `race.c`

In `race.c`, two threads (`a` and `b`) each execute a loop of $N = 1\,000\,000$ iterations incrementing the shared global variable `counter` without any synchronization:

```
/* race.c - two threads add to the same counter, with no protection.
 * Build: gcc -Wall -pthread -o race race.c   (no -O2: keep counter++ in memory)
 * Run:   ./race   (several times)
 */
#include <stdio.h>
#include <pthread.h>

#define N 1000000

long counter = 0;                /* shared by both threads */

void *add(void *arg) {
    for (int i = 0; i < N; i++)
        counter++;               /* read, add 1, write back: three steps */
    return NULL;
}

int main(void) {
    pthread_t a, b;
    pthread_create(&a, NULL, add, NULL);
    pthread_create(&b, NULL, add, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("counter = %ld (expected %d)\n", counter, 2 * N);
    return 0;
}
```

Compiling without `-O2` (so the compiler loads and stores `counter` in memory on every iteration) and running `./race` three times produced:

```
$ gcc -Wall -pthread -o race race.c
$ ./race
counter = 1031820 (expected 2000000)
$ ./race
counter = 1321476 (expected 2000000)
$ ./race
counter = 1153608 (expected 2000000)
```

1.2 Observations for Part 1

1. **`counter++` looks like one step. Write the three steps the CPU really does.** At the hardware level, incrementing a variable in memory without an atomic prefix executes three separate operations:
   (a) *Load (Read):* Copy the current value of `counter` from memory into a CPU register (e.g., `movq counter(%rip), %rax`).
   (b) *Add (Modify):* Add `1` to the register value inside the CPU's arithmetic logic unit (e.g., `addq`

`$1, %rax`).

(c) *Store (Write):* Write the updated register value back to `counter` in memory (e.g., `movq %rax, counter(%rip)`).

2. **Describe, with those steps, how two threads can lose one addition.** Suppose `counter` in memory currently equals `100`:
   - Thread A executes **Step 1 (Load)** and reads `100` into its register.
   - Simultaneously on another core (or after a context switch before Thread A stores), Thread B executes **Step 1 (Load)** and also reads `100` into its own register.
   - Thread A executes **Step 2 (Add)** ($100+1 = 101$) and **Step 3 (Store)**, writing `101` to `counter`.
   - Thread B executes **Step 2 (Add)** ($100 + 1 = 101$) and **Step 3 (Store)**, overwriting `counter` with `101`.

   Both threads incremented the value they loaded, yet `counter` advanced from `100` to `101` instead of `102`, losing one addition.
3. **Why is the result different at every run?** Thread scheduling across CPU cores, timer interrupts, cache-line invalidations, and background OS activity vary slightly on every run. Because the exact interleaving of the two threads' load-add-store instructions is non-deterministic, the number of overlapping increments changes every time `./race` is executed (`1031820`, `1321476`, `1153608`).

2 PART 2: FIX IT: MUTEX, THEN SEMAPHORE

2.1 2.1 A mutex around `counter++` (`mutex.c`)

Placing `pthread_mutex_lock(&lock)` before `counter++` and `pthread_mutex_unlock(&lock)` after it turns `counter++` into a critical section that only one thread may execute at a time:

```
/* mutex.c - the same counter, now protected by a mutex.
 * Build: gcc -Wall -pthread -o mutex mutex.c
 */
#include <stdio.h>
#include <pthread.h>

#define N 1000000

long counter = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *add(void *arg) {
    for (int i = 0; i < N; i++) {
        pthread_mutex_lock(&lock);    /* only one thread past this line */
        counter++;                    /* the critical section */
        pthread_mutex_unlock(&lock);  /* let the other one in */
    }
    return NULL;
}

int main(void) {
    pthread_t a, b;
    pthread_create(&a, NULL, add, NULL);
    pthread_create(&b, NULL, add, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("counter = %ld (expected %d)\n", counter, 2 * N);
    return 0;
}
```

2.2 2.2 The same lock, made from a semaphore (`sem.c`)

A POSIX semaphore initialized to 1 (`sem_init(&s, 0, 1)`) behaves as a binary lock: `sem_wait(&s)` decrements the value from `1` to `0` (or blocks when it is already `0`), and `sem_post(&s)` increments it back from `0` to `1`:

```
/* sem.c - the same counter, protected by a semaphore that starts at 1.
 * Build: gcc -Wall -pthread -o sem sem.c
 */
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define N 1000000

long counter = 0;
sem_t s;
```

```c
void *add(void *arg) {
    for (int i = 0; i < N; i++) {
        sem_wait(&s);                 /* 1 -> 0: I go in; at 0: wait */
        counter++;
        sem_post(&s);                 /* 0 -> 1: the next one may go in */
    }
    return NULL;
}

int main(void) {
    sem_init(&s, 0, 1);               /* 0 = shared between threads, 1 = start value */
    pthread_t a, b;
    pthread_create(&a, NULL, add, NULL);
    pthread_create(&b, NULL, add, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("counter = %ld (expected %d)\n", counter, 2 * N);
    sem_destroy(&s);
    return 0;
}
```

Running both `./mutex` and `./sem` three times each:

```
$ gcc -Wall -pthread -o mutex mutex.c
$ ./mutex
counter = 2000000 (expected 2000000)
$ ./mutex
counter = 2000000 (expected 2000000)
$ ./mutex
counter = 2000000 (expected 2000000)

$ gcc -Wall -pthread -o sem sem.c
$ ./sem
counter = 2000000 (expected 2000000)
$ ./sem
counter = 2000000 (expected 2000000)
$ ./sem
counter = 2000000 (expected 2000000)
```

2.3 2.3 The price of the lock

Measuring the wall-clock, user, and kernel CPU time of `./race`, `./mutex`, and `./sem`:

```
$ time ./race
counter = 1125516 (expected 2000000)
real 0.015s  user 0.025s  sys 0.001s

$ time ./mutex
counter = 2000000 (expected 2000000)
real 0.157s  user 0.195s  sys 0.104s

$ time ./sem
counter = 2000000 (expected 2000000)
real 0.454s  user 0.557s  sys 0.344s
```

2.4 Observations for Part 2

1. **Run `mutex` and `sem` three times each. Is the answer always 2 000 000? Yes.** Across all three runs of `./mutex` and all three runs of `./sem`, the output is always `counter = 2000000 (expected 2000000)`. Mutual exclusion ensures that the three-step load-add-store sequence of one thread never overlaps with that of the other.
2. **What would happen in `sem.c` with `sem_init(&s, 0, 2)`? Try it and explain.** Changing the initial semaphore value to 2 and running three times produced:

   ```
   counter = 1983089 (expected 2000000)
   counter = 1989142 (expected 2000000)
   counter = 1988601 (expected 2000000)
   ```

   The final count falls short of 2 000 000 again. Starting the semaphore at 2 permits **two threads** to pass `sem_wait(&s)` concurrently (the first decrements `s` from 2 to 1, and the second decrements it from 1 to 0 without blocking). Because there are only two threads in the program, neither thread ever blocks and both enter the critical section simultaneously, recreating the data race.
3. **Why is the locked version slower?** Whereas `./race` finishes in `0.015s` (with a wrong answer), `./mutex` takes `0.157s` and `./sem` takes `0.454s` for three reasons:
   - *Atomic operations and cache coherence:* Acquiring and releasing the lock 2 000 000 times executes hardware atomic read-modify-write instructions and memory fences that transfer ownership of the lock's cache line back and forth between CPU cores.
   - *Kernel `futex` contention and context switching:* When one thread attempts to lock while the other thread holds the lock, it invokes the Linux `futex` system call to sleep and wake up in the kernel, increasing system CPU time from `0.001s` in `race` to `0.104s` in `mutex` and `0.344s` in `sem`.
   - *Serialization:* Only one thread can execute inside the critical section at any instant.

3 PART 3: A SEMAPHORE AS A SIGNAL

3.1 Source code and execution of `order.c`

In `order.c`, thread B (`reader`) is started before thread A (`writer`), but must print `message` only after thread A has written `"hello"`. The semaphore `ready` is initialized to `0`:

```
/* order.c - thread B must wait until thread A has written the message.
 * A semaphore that starts at 0 is a signal: sem_wait blocks until someone posts.
 * Build: gcc -Wall -pthread -o order order.c
 */
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

char message[32];
sem_t ready;                          /* starts at 0: nothing is ready yet */

void *writer(void *arg) {
    sleep(1);                         /* pretend the work takes a while */
    strcpy(message, "hello");
    printf("A: message written\n");
    sem_post(&ready);                 /* 0 -> 1: wake up B */
    return NULL;
}

void *reader(void *arg) {
    printf("B: waiting for the message...\n");
    sem_wait(&ready);                 /* blocks while the value is 0 */
    printf("B: I read \"%s\"\n", message);
    return NULL;
}

int main(void) {
    sem_init(&ready, 0, 0);
    pthread_t a, b;
    pthread_create(&b, NULL, reader, NULL);   /* B starts first, on purpose */
    pthread_create(&a, NULL, writer, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    sem_destroy(&ready);
    return 0;
}
```

Compiling and running `./order`:

```
$ gcc -Wall -pthread -o order order.c
$ ./order
B: waiting for the message...
A: message written
B: I read "hello"
```

3.2 Observations for Part 3

1. **Change the start value to 1 (`sem_init(&ready, 0, 1)`), compile, run. What does B read now, and why?** Running with `sem_init(&ready, 0, 1)` produces:

   ```
   B: waiting for the message...
   B: I read ""
   A: message written
   ```

   Thread B reads the empty string `""` before Thread A writes `"hello"`. Because `ready` starts at `1`, `sem_wait(&ready)` in `reader()` immediately decrements `ready` from `1` to `0` and returns without waiting. Thread B therefore prints the global `message` buffer (initially all zero bytes) while Thread A is still sleeping in `sleep(1)`.
2. **Put it back to 0. In one sentence: what does a semaphore that starts at 0 mean?** A semaphore initialized to `0` represents a **synchronization signal for an event that has not yet occurred**, causing any waiting thread (`sem_wait`) to block until another thread announces the event (`sem_post`).

4 PART 4: A SEMAPHORE AS A COUNTER: THE PARKING LOT

4.1 Source code and execution of `parking.c`

In `parking.c`, six car threads compete for three parking spaces. The semaphore `places` starts at `PLACES` (3) and counts how many free spaces remain, while `lock` protects the `inside` counter and `printf` call:

```
/* parking.c - 6 cars, 3 places. A semaphore that starts at 3 counts the free places.
 * Build: gcc -Wall -pthread -o parking parking.c
 */
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

#define PLACES 3
#define CARS   6

sem_t places;                           /* free places */
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
int inside = 0;                         /* only for printing */

void *car(void *arg) {
    long id = (long)arg;
    sem_wait(&places);                  /* take a place, or wait */
    pthread_mutex_lock(&lock);
    inside++;
    printf("car %ld enters (inside: %d)\n", id, inside);
    pthread_mutex_unlock(&lock);

    sleep(1);                           /* parked */

    pthread_mutex_lock(&lock);
    inside--;
    printf("car %ld leaves (inside: %d)\n", id, inside);
    pthread_mutex_unlock(&lock);
    sem_post(&places);                  /* give the place back */
    return NULL;
}

int main(void) {
    sem_init(&places, 0, PLACES);
    pthread_t t[CARS];
    for (long i = 0; i < CARS; i++)
        pthread_create(&t[i], NULL, car, (void *)(i + 1));
    for (int i = 0; i < CARS; i++)
        pthread_join(t[i], NULL);
    sem_destroy(&places);
    return 0;
}
```

Compiling and running `./parking`:

```
$ gcc -Wall -pthread -o parking parking.c
$ ./parking
car 1 enters (inside: 1)
```

```
car 2 enters (inside: 2)
car 3 enters (inside: 3)
car 1 leaves (inside: 2)
car 3 leaves (inside: 1)
car 2 leaves (inside: 0)
car 4 enters (inside: 1)
car 5 enters (inside: 2)
car 6 enters (inside: 3)
car 4 leaves (inside: 2)
car 5 leaves (inside: 1)
car 6 leaves (inside: 0)
```

4.2 Observations for Part 4 and Optional Atomic Counter

1. **What is the largest value of `"inside"` you see? Which line of code guarantees it?** The largest value of `inside` is `3`. This upper bound is guaranteed by `sem_wait(&places);` at the start of `car()` (together with `sem_init(&places, 0, PLACES);` where `PLACES` is `3`): after three cars decrement `places` from `3` down to `0`, any additional car thread calling `sem_wait(&places)` blocks until a parked car leaves and calls `sem_post(&places)`.
2. **Set `PLACES` to 1. What does the semaphore become?** With `#define PLACES 1`, the output becomes strictly one car at a time (`inside` never exceeds `1`):

   ```
   car 1 enters (inside: 1)
   car 1 leaves (inside: 0)
   car 2 enters (inside: 1)
   car 2 leaves (inside: 0)
   ...
   car 6 enters (inside: 1)
   car 6 leaves (inside: 0)
   ```

   Setting `PLACES` to `1` turns the counting semaphore into a **binary semaphore (a mutex / lock)** that enforces mutual exclusion so only one car can be inside the parking lot at a time.

Finally, the optional lock-free program `atomic.c` replaces `long counter` with `atomic_long counter` and increments it via `atomic_fetch_add(&counter, 1)`:

```
$ gcc -Wall -pthread -o atomic atomic.c
$ time ./atomic
counter = 2000000 (expected 2000000)
real 0.080s  user 0.154s  sys 0.001s
```

Because `atomic_fetch_add` compiles to a single hardware-locked instruction (`lock xaddq`), it eliminates the race without any kernel `futex` calls (`sys 0.001s`), running in `0.080s` compared with `0.157s` for `mutex` and `0.454s` for `sem`.

CONCLUSIONS

This laboratory demonstrated why unsynchronized concurrent updates fail and how a single synchronization primitive—the semaphore—can solve three distinct problems depending on its initial value:

- In **Part 1**, running `race.c` showed that `counter++` is a three-step load-add-store sequence in which concurrent threads overwrite each other's updates non-deterministically.
- In **Part 2**, protecting the critical section with either `pthread_mutex_t` or a semaphore initialized to **1** restored the exact sum of `2 000 000`, whereas initializing the semaphore to 2 allowed both threads inside at once and reintroduced lost updates.
- In **Part 3**, initializing a semaphore to **0** (`order.c`) turned it into an ordering signal that blocked the reader until the writer posted completion.
- In **Part 4**, initializing a semaphore to **3** (`parking.c`) turned it into a resource counter that bounded concurrency to at most three threads inside the critical region simultaneously.